Architectural Evolution of the Stela System: Distributed Provenance, Ledger Integration, and Operational Hardening

1. Executive Summary and Constitutional Posture

The Stela system currently stands at a critical architectural inflection point. The repository operates under a rigid "Policy of Truth" (PoT) that mandates a single source of truth (SSoT) for every domain, enforced by a suite of policing workflows and linting gates.[1] However, the current implementation of this governance—relying on monolithic, manually updated root ledgers (PoW.md and SoP.md)—is increasingly at odds with the scalability and automation requirements of a modern distributed system. The proposed transition to a "Provenance Leaf" architecture, utilizing a distributed "Index-Leaf" logging model, presents both a profound opportunity to strengthen the system's constitutional integrity and a significant risk of disrupting the established Filing Doctrine.

This report provides an exhaustive analysis of the constitutional, structural, and operational implications of this transition. It concludes that integrating the Proof of Work (PoW) and State of Play (SoP) ledgers into a distributed logging system **strengthens** the Policy of Truth by moving from a model of "asserted truth" (manual entry) to "proven truth" (system-generated artifacts). However, this conclusion is contingent upon a rigorous amendment of the Filing Doctrine to reclassify storage/archives/ from its current status as "Trash" to a protected "Immutable Store".[1]

Furthermore, the forensic audit conducted for this report reveals a fragile operational substrate. The ops/ and tools/ directories are riddled with hardcoded paths, overloaded binary logic, and unused code that threaten the stability of any architectural migration.[1] A concurrent hardening plan is therefore not merely an optimization but a prerequisite for the safe adoption of the distributed ledger model.

2. Constitutional Analysis: The Policy of Truth in a Distributed Context

The "Policy of Truth" (PoT.md) serves as the constitution for the Stela repository. It defines the physical laws, axioms, and staffing protocols that govern the system.[1] To determine whether the proposed distributed logging architecture weakens or strengthens this constitution, we must dissect the specific axioms defined in PoT.md and evaluate their resilience against the "Index-Leaf" model.

2.1 The Axiom of the Single Source of Truth (SSoT)

The SSoT axiom is the cornerstone of Stela's governance. It states: "one canonical file per domain; other mentions are pointers".[1] Currently, PoW.md acts as the SSoT for execution history, and SoP.md acts as the SSoT for project state history. These are monolithic files residing at the repository root.

2.1.1 The Fragility of the Monolith

In the current monolithic model, the "Truth" is contained within a single mutable file. This presents several constitutional risks:

1. **Concurrency Contention:** As multiple agents or contractors attempt to log work simultaneously, the monolithic file becomes a bottleneck, leading to merge conflicts. While git handles text merges, the semantic integrity of an ordered log can be compromised.
2. **Policing Brittleness:** The enforcement of SSoT for PoW.md relies on the .github/workflows/pow_policing.yml workflow, which uses fragile logic to detect changes. It explicitly checks for the presence of PoW.md in the git diff output.[1] If the file is renamed or if the policing logic fails to parse a non-standard diff header, the SSoT protection evaporates.
3. **Growth Unbounded:** An append-only text file grows indefinitely. Over time, parsing PoW.md to verify the existence of a specific packet ID becomes computationally expensive and prone to timeout errors in CI environments.

2.1.2 The Strength of the Index-Leaf Model

The proposed "Provenance Leaf" architecture distributes the "data" of the truth into individual, immutable leaf files (e.g., storage/archives/logs/trace_UUID.md) while treating the root ledger (PoW.md) as an "Index" or view.[1]

- **Constitutional Alignment:** Does this violate SSoT? No. It refines it. In database theory, the SSoT is the *record*, not the *view*. By making the Leaf the atomic unit of record, we ensure that the specific execution event is immutable and isolated. The Index (PoW.md) becomes a derived pointers file.
- **Pointer-First Discipline:** The proposal aligns perfectly with the "Pointer-First" logic observed in TASK.md updates.[1] The Index points to the Leaf. This adheres to the axiom "other mentions are pointers."
- **Conclusion:** The distributed model **strengthens SSoT** by decoupling the *storage* of truth (the immutable leaf) from the *presentation* of truth (the index). This eliminates the risk of accidental history rewriting during merge conflicts, as Leaves are never edited, only created.

2.2 The Filing Doctrine and the "Trash" Paradox

PoT.md Section 1.1 defines the Filing Doctrine, establishing the legal status of directories:

- ops/ = Run
- docs/ = Explain
- projects/ = Work
- opt/ = Isolate
- storage/ = **Trash** (local artifacts, never canon).[1]

This definition creates a constitutional crisis for the proposed architecture. The proposal suggests moving PoW and SoP data—which are high-value Canon—into storage/archives/. Under the current law, this effectively designates the project's history as "Trash."

2.2.1 The "Trash" Designation Risk

If storage/ is defined as "Trash" and "never canon," then placing provenance leaves in that directory strips them of their governance weight. A hostile actor could delete storage/archives/ and claim they were merely "taking out the trash" in accordance with PoT.md. Furthermore, tools/verify.sh likely enforces hygiene rules that might aggressively prune "Trash" directories [1], putting the history at risk.

2.2.2 The Necessity of Amendment

To adopt the distributed model without weakening the Policy of Truth, PoT.md must be amended. We cannot simply place Canon in Trash.

- **Recommendation:** Redefine storage/ or create a new namespace. The most elegant solution is to bifurcate storage/:
  - storage/tmp/: Remains "Trash" (ephemeral).
  - storage/archives/: Redefined as **"The Immutable Store"** or "Museum." This subsection of storage must be elevated to Canon status, protected by the same policing rules as docs/.
- **Alternative:** Move the archives to docs/history/. However, docs/ is defined as "Explain" (human-readable manuals). Automated log leaves are "Evidence," not "Explanation." Therefore, redefining storage/archives/ is the philosophically correct path.

2.3 The Axiom of Drift and Provenance

"Drift" is defined as any divergence between canon and repository state.[1] Currently, provenance is asserted manually. An operator runs a command and *claims* to have verified it by writing an entry in PoW.md.

2.3.1 The Fallibility of Manual Entry

The current pow_policing workflow checks *if* PoW.md was updated, but it cannot verify *what* was written.[1] An operator could write "Updated logging" while actually injecting malicious code. The "Proof" is decoupled from the "Work."

2.3.2 The Assurance of Machine-Generated Leaves

The "Markdown-Header Schema" [1] introduces strict fields: TraceID, Context, State Check, and Provenance (link to binary).

- **Mechanism:** If the tools (ops/bin/*) are refactored to auto-generate these Leaves upon successful execution, the provenance becomes intrinsic to the action. You cannot generate a "Success" Leaf if the tool failed.
- **Constitutional Gain:** This minimizes Drift. The repository state (the Leaf) is a direct, causal result of the execution. The gap between "what happened" and "what is recorded" closes.
- **Conclusion:** Integrating this system **strengthens the Truth axiom**. It shifts the burden of truth from human honesty to cryptographic/algorithmic certainty.

---

3. Structural Analysis of the Existing Logging Architecture

Before designing the integration plan, we must perform a forensic audit of the current logging functions (Factory, PoW, SoP) to understand their mechanisms and constraints.

3.1 The Factory Registry (opt/_factory/)

The opt/_factory/ directory is the most mature implementation of a distributed registry in the current system. It already functions as a primitive Index-Leaf system.

- **Structure:**
  - **Indices:** AGENTS.md, TASKS.md, SKILLS.md act as the root ledgers.
  - **Leaves:** Individual files in agents/, tasks/, skills/ (e.g., R-AGENT-01.md) act as the data definition files.[1]
- **Mechanism:**
  - Scripts ops/lib/scripts/agent.sh, task.sh, and skill.sh manage the creation of these files.[1]
  - **Append Logic:** These scripts utilize a direct cat >> or echo >> approach to append a new table row to the Index file immediately after creating the Leaf.[1]
  - **Formatting:** The appended lines follow a strict Markdown table format: |(path) | Name |... |.
- **Weakness:** The append logic is tightly coupled to the creation logic. If the echo >> fails or causes a merge conflict, the Leaf exists (orphaned) without an Index entry. This violates SSoT because the Leaf exists but is not "registered."

3.2 The Proof of Work Ledger (PoW.md)

PoW.md is the "Code Surface" ledger. Its integrity is heavily policed.

- **Structure:** A flat, append-only Markdown file. Entries are chronological.
- **Policing Logic (pow_policing.yml):**
  - **Trigger:** Regex ^(ops/|tools/|projects/|\.github/workflows/) defines the "Code Surface".[1]
  - **Integrity:** Uses git diff --unified=0... -- PoW.md | grep -E '^-' to detect deletions.[1] This is a "hard gate." Any deletion causes a CI failure.
- **Workflow:** Currently manual. The developer edits code, then manually adds a block to PoW.md summarizing the work.
- **Brittleness:** The reliance on grep -Ev '^--- a/PoW\.md$' to ignore the diff header is extremely fragile. Changes in git versions or diff formats could trigger false positives.[1]

3.3 The State of Play Ledger (SoP.md)

SoP.md is the "Canon" ledger.

- **Structure:** Narrative history ("what + why").
- **Policing Logic (sop_policing.yml):**
  - **Trigger:** Changes to PoT.md, TASK.md, docs/, or .github/copilot-instructions.md.[1]
  - **Exceptions:** Hardcoded carve-out for docs/ops/log/.[1] This is a critical precedent. It proves the system *can* tolerate files that do not trigger SoP updates.
- **Workflow:** Manual. Required for "Audit Readiness".[1]

3.4 Operational Gaps

The audit reveals a significant gap in automation. While factory scripts automate their own registration, PoW and SoP updates are entirely manual. This introduces:

1. **Human Error:** Typos in PoW packet IDs.
2. **Timing Drifts:** Updates to SoP that happen in a different PR than the Canon change.
3. **Lack of Linkage:** PoW entries rarely link deeply to the specific file diffs or logs, offering only a high-level summary.

---

4. Integration Plan: The Unified Distributed Logging System

This section outlines the strategic plan to integrate all logging functions into the "Provenance Leaf" model, treating root ledgers as Indices.

4.1 Architectural Definitions

We define the new components of the logging architecture:

- **Provenance Leaf:** An immutable Markdown file generated at the moment of execution. Stored in storage/archives/logs/<YEAR>/<MONTH>/<TRACE_ID>.md.

    - *Schema:* Must contain YAML Front-Matter (session_id, governance_mode) and standard Headers (TraceID, Context, State Check, Provenance).[1]
- **Root Index:** A dynamic file (PoW.md, SoP.md) that aggregates data from the Leaves. It serves as the human-readable view.
- **The Indexer:** A new binary (ops/bin/index) responsible for scanning Leaves and regenerating the Root Indices.

4.2 Phase 1: Factory Integration (The Pilot)

The Factory is the ideal pilot because it already uses a primitive Index-Leaf model.

Step 1: Standardize Factory Leaves

- Modify ops/src/definitions/agent.md.tpl (and skill/task templates) to include the **Markdown-Header Schema**.
- Add YAML front-matter: type: agent, id: R-AGENT-XX, status: active.
- Add Provenance footer: Generated by: ops/lib/scripts/agent.sh.

Step 2: Decouple Registration

- Modify ops/lib/scripts/agent.sh (and counterparts).
- **Remove:** The cat >> opt/_factory/AGENTS.md logic.[1]
- **Action:** The script now *only* creates the Leaf file (opt/_factory/agents/R-AGENT-XX.md) and validates it. It does *not* touch the Index.

Step 3: Implement Dynamic Indexing

- Develop ops/bin/index.
- *Logic:*
    1. Scan opt/_factory/agents/*.md.
    2. Extract metadata (ID, Name, Status) from the YAML front-matter.
    3. Sort by ID.
    4. Regenerate opt/_factory/AGENTS.md completely.
- **Outcome:** The Index is now a guaranteed reflection of the Leaves. "Orphaned" agents are impossible.

4.3 Phase 2: PoW Integration (The "Append-Only" Challenge)

Moving PoW.md to an Index-Leaf model is complex because of the pow_policing.yml "Append-Only" constraint. If ops/bin/index regenerates the file, it might look like a rewrite (deletion + addition) to the CI.

Strategy: The Hybrid Pointer Model

Step 1: Automate Leaf Generation

- Create a wrapper script ops/lib/scripts/logger.sh.

- Update all ops/bin tools to call logger.sh on execution.
- logger.sh generates a file: storage/archives/logs/YYYY/MM/PoW_Leaf_<TraceID>.md.
- Content includes the command run, the user, the output, and the git hash.

Step 2: The Indexer Bridge

- The ops/bin/index tool scans storage/archives/logs/ for *new* leaves that are not yet in PoW.md.
- It *appends* a summary line for each new leaf to PoW.md.
  - Format: | <TRACE_ID> | <COMMAND> | [Link to Leaf]
- **Why Append?** To satisfy pow_policing.yml. By appending lines pointing to the leaves, we respect the "no deletions" rule while offloading the heavy data to the Leaf.

Step 3: Update Policing

- Refactor pow_policing.yml to recognize the new Leaf directory.
- New Trigger: Changes in ops/, tools/ OR creation of file in storage/archives/logs/.
- New Check: Ensure PoW.md contains a link to the new Leaf.

4.4 Phase 3: SoP Integration (The "Intent" Layer)

SoP tracks *rationale* ("why"). This requires human input.

Step 1: The SoP Leaf

- Create a template ops/src/surfaces/sop_entry.md.tpl.
- Fields: Objective, Rationale, Impact, Canon_Modified.

Step 2: Authoring Workflow

- When a developer changes Canon, they create a new file: storage/archives/sop/YYYY/MM/SoP_Leaf_<ShortDesc>.md.
- They write the "Why" in this file.

Step 3: Indexing

- ops/bin/index scans these SoP Leaves.
- It appends a summary to the root SoP.md.
- **Benefit:** The root SoP.md remains a clean, high-level timeline. The detailed arguments and context are preserved in the immutable Leaves.

4.5 Managing the storage/archives Namespace

To support this, storage/archives must be rigorously structured and protected.

Proposed Structure:

storage/

├── archives/

│ ├──.gitkeep

│ ├── root/ # Historical archives of the monoliths

│ ├── logs/ # PoW Leaves (Machine Generated)

│ │ └── YYYY/

│ │ └── MM/

│ ├── sop/ # SoP Leaves (Human Generated)

│ │ └── YYYY/

│ │ └── MM/

│ └── factory/ # Retired agents/tasks

Regulatory Adjustment:

- Update PoT.md Filing Doctrine to explicitly classify storage/archives/ as **Canon-Adjacent**.
- Update .gitignore to *allow* storage/archives/ while ignoring storage/tmp/.[1]

---

5. Concurrent Plan: Reducing Code Brittleness

The repository audit identified severe brittleness in the ops/ and tools/ directories that makes the system fragile and resistant to change. This section details the remediation plan.

5.1 Identification of Overloaded Scripts

Two binaries were identified as "Dispatchers/Multiplexers" that violate the separation of concerns.[1]

1. ops/bin/project

- **Current Role:** Orchestrator for project lifecycle.
- **Overload:** It handles Scaffolding (init), Discovery (ls), and Configuration (set).
- **Risk:** A bug in the scaffolding template logic breaks the ls command, blinding the operator to existing projects.
- **Action:** Split into atomic binaries.
  - ops/bin/scaffold: Dedicated to creating new projects/structures.

- ops/bin/inventory: Dedicated to discovery (listing projects, agents, tasks).
- ops/bin/config: Dedicated to setting state variables.

2. ops/bin/template

- **Current Role:** Template instantiation engine.
- **Overload:** Handles Synthesis (apply) and Inventory (ls).
- **Risk:** Duplicates the discovery logic found in Factory scripts.
- **Action:** Strip the ls capability. ops/bin/template should strictly be a renderer (input tpl -> output md). Move listing logic to ops/bin/inventory.

5.2 Identification and Remediation of Hardcoded Paths

The forensic audit of workflows revealed dangerous hardcoded paths.[1]

Table 1: Hardcoded Path Inventory and Remediation

Location	Hardcoded Path	Risk	Remediation
pow_policing.yml	^(ops/|tools/|projects/|...	New code dirs (e.g. src/) ignored by policing.	Move regex to ops/config.json. Script reads config.
pow_policing.yml	PoW.md	Breaking change if ledger is renamed.	Define LEDGER_POW env var in workflow.
repo_gates.yml	tools/lint/dp.sh TASK.md	Fails if TASK.md moves or linter moves.	Use variable $ACTIVE_WORK_SURFACE.
sop_policing.yml	docs/ops/log/	"Museum logs" carve-out is brittle.	Define EXCLUDE_PATHS in repo config.
verify.sh	PoT.md, SoP.md, etc.	Verification fails on valid rename.	Read mandatory list from ops/config.json.
task.sh	opt/_factory/tasks	Factory move breaks automation.	Source ops/lib/manifests/PATHS.env.

Strategic Fix: Centralized Configuration

Create ops/lib/manifests/PATHS.env (or .json) as the SSoT for repository paths.

- All scripts (verify.sh, agent.sh) must source this file instead of using string literals.
- Workflows can read this via a setup step (e.g., jq) to set environment variables.

5.3 Elimination of Unused and Redundant Code

A comparison of the file index against CI execution logs reveals "Dead Code" that creates noise and confusion.[1]

Unused Tools (Immediate Removal Candidates):

The following tools exist in tools/lint/ but are **never** called by repo_gates.yml or other workflows:

1. **tools/lint/agent.sh**: Redundant. Logic likely belongs in ops/lib/scripts/agent.sh.
2. **tools/lint/context.sh**: Unused.
3. **tools/lint/llms.sh**: Unused. LLM generation handles its own validation.
4. **tools/lint/project.sh**: Unused.
5. **tools/lint/style.sh**: Unused.
6. **tools/test/agent.sh**: Unused.

Redundant Logic (Consolidation Candidates):

- **tools/lint/task.sh**: The workflow uses tools/lint/dp.sh to lint TASK.md. task.sh appears to be a legacy or duplicate script.
    - **Action:** Audit task.sh for unique logic. Merge into dp.sh if valuable. Delete task.sh.

Utility Function Standardization:

While the specific script content is missing, the "Drift Mitigation" instruction implies a need to standardize utilities.

- **Action:** Create ops/lib/utils/common.sh.
- **Functions:** Implement robust die(), log(), require_file() functions here.
- **Refactor:** Update all ops/bin and ops/lib/scripts to source common.sh. This prevents the "copy-paste" drift of error handling logic.

---

6. Implementation Roadmap

This transformation should be executed in three distinct phases to maintain stability.

Phase 1: Hygiene and Stabilization (Week 1)

- **Objective:** Clean the workspace and centralize configuration.
- **Tasks:**
  1. Delete the 7 unused scripts in tools/lint/.
  2. Create ops/lib/manifests/PATHS.env defining all repository constants.
  3. Refactor tools/verify.sh and pow_policing.yml to use these constants (or env vars).
  4. Update PoT.md to redefine storage/archives/ as "The Immutable Store."

Phase 2: The Distributed Foundation (Week 2)

- **Objective:** Implement the Leaf generation logic.
- **Tasks:**
  1. Create ops/lib/scripts/logger.sh implementing the Markdown-Header Schema.
  2. Update Factory scripts (agent.sh, task.sh) to use logger.sh and stop appending to root files.
  3. Create the factory templates with YAML front-matter.

Phase 3: The Index Pivot (Week 3)

- **Objective:** Switch from Manual Appends to Dynamic Indexing.
- **Tasks:**
  1. Develop ops/bin/index.
  2. Run the indexer to regenerate opt/_factory/*.md.
  3. Update pow_policing.yml to allow indexer-based updates.
  4. Split ops/bin/project and template into atomic tools.

7. Conclusion

The transition to a distributed "Index-Leaf" architecture is a necessary evolution for the Stela system. It resolves the constitutional tension of the monolithic ledger, enforcing Truth through cryptographic linkage rather than human assertion. By treating the Root Ledgers as dynamic views of immutable Provenance Leaves, the system achieves a higher order of auditability and resilience.

However, this sophistication requires a solid foundation. The current codebase—fragile, hardcoded, and partially redundant—cannot support this weight without the prescribed remediation. By executing the "Hygiene, Foundation, Pivot" roadmap, Stela can mature from a managed repository into a truly autonomous governance engine.

Recommendations:

1. **Authorize the PoT.md Amendment:** Redefine storage/archives/ immediately.
2. **Execute the Purge:** Remove dead code to clear the path for refactoring.
3. **Adopt the Schema:** Standardize all tool outputs to the Markdown-Header Schema before attempting distributed storage.

This path ensures that the "Policy of Truth" remains not just a document, but an operational reality.

Works cited

1. dump-full-main-80bb3afd.txt